PROYECTO INTEGRADOR

PrestaLab

Sistema de reserva y control de préstamo de equipo y espacios de laboratorio

ENTREGA 3 — Diseño de la API y contratos de servicio

Carrera	Ingeniería en Software
Materia	Arquitectura Orientada a Servicios
Docente	Ing. Luis Leonardo Mendoza Reséndiz
Equipo	Tranquilo nos la navegamos
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Periodo / Semana	[Periodo] · Semana 3 de 8
Repositorio	https://github.com/JhonaRaida/prestalab · tag: entrega-3

Reporte breve de la entrega

Entrega	3 — Diseño de la API y contratos de servicio
Nombre del proyecto	PrestaLab (nombre provisional)
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Objetivo de la semana	Definir con precisión cómo serán consumidos los servicios: recursos, rutas, métodos, parámetros, contratos JSON, códigos HTTP, errores, autenticación y versionado.
Qué se diseñó	12 recursos, 33 endpoints públicos y 4 rutas internas entre servicios, 10 contratos JSON con ejemplos, 12 códigos HTTP explicados, formato único de errores con 17 códigos de aplicación y 8 ejemplos, autenticación con JWT, versionado /api/v1, 2 diagramas de flujo y un borrador de OpenAPI validado.
Qué cambió respecto a la entrega anterior	La arquitectura de la Entrega 2 no cambió. Se cerraron decisiones que quedaban abiertas: la disponibilidad se expone como recurso derivado (/disponibilidad), las acciones (iniciar sesión, entregar, devolver, cancelar) se modelan como recursos o cambios de estado, se definieron cuatro rutas internas entre servicios y los identificadores serán enteros.
Problemas encontrados	Cómo representar como recursos operaciones que parecen acciones (iniciar sesión, entregar, devolver, cancelar); cuándo usar 400, 409 o 422; cómo permitir que los servicios se llamen entre sí sin exponer esas rutas al exterior; y cómo evitar que los errores revelen información interna.
Cómo se resolvieron	Sesiones, préstamos y reservas se modelaron como sustantivos, con POST para crear y PATCH para cambiar su estado. Se fijó una regla: 400 petición mal formada, 409 choque con el estado actual, 422 regla de negocio incumplida. Las rutas internas viven bajo /internal/v1, el Gateway no las publica y exigen un token de servicio. Los errores usan un formato único con código estable y sin detalles internos.
Pendientes para la semana 4	Modelo de datos: diagrama entidad-relación, esquema físico, scripts SQL de creación, diccionario de datos, datos de prueba (mínimo 10 registros) y documento que relacione cada servicio con sus datos.
Enlace al repositorio y versión	https://github.com/JhonaRaida/prestalab · tag: entrega-3

1. Alcance y convenciones generales

Este documento define el contrato público de la API de PrestaLab, es decir, lo que debe enviar y recibir cualquier consumidor (Postman, Swagger UI o una futura aplicación). Parte de los 10 requisitos funcionales, los roles y los 5 servicios de la Entrega 2. Todas las rutas se publican a través del API Gateway.

Convención	Decisión
URL base	http://localhost:8080/api/v1 (Gateway en Docker Compose local).
Formato	JSON en UTF-8. Encabezado Content-Type: application/json.
Rutas	Sustantivos en plural y en minúsculas, sin verbos. Un solo nivel de anidamiento (por ejemplo, /usuarios/{usuarioId}/sanciones).
Acciones	Se modelan como recursos o cambios de estado: iniciar sesión es crear una sesión; entregar es crear un préstamo; devolver y cancelar son PATCH sobre el recurso.
Métodos	GET consulta y no modifica. POST crea. PUT reemplaza un recurso completo. PATCH modifica parte de un recurso. DELETE da de baja (baja lógica) y es idempotente.
Nombres de campos	camelCase. Las enumeraciones van en minúsculas, sin acentos ni ñ (por ejemplo, «dano» y «danado»).
Fechas	ISO 8601 con zona horaria, por ejemplo 2026-11-03T10:00:00-06:00.
Identificadores	Enteros positivos asignados por el servidor.
Paginación	Parámetros page (por defecto 1) y limit (por defecto 20, máximo 100). Las listas responden { "data": [...], "meta": { page, limit, total } }.
Filtros	Por parámetros de consulta (query string).
Respuestas de creación	201 Created, con el recurso en el cuerpo y su URL en el encabezado Location.
Versionado	Prefijo /api/v1. Agregar campos opcionales o endpoints no cambia la versión. Un cambio incompatible (quitar o renombrar campos, cambiar el significado de un código) se publica como /api/v2 y v1 se mantiene durante un periodo de transición.
Trazabilidad	El Gateway asigna un identificador a cada petición (idPeticion) que aparece en los errores y en los registros.

2. Autenticación y autorización prevista

- Rutas públicas: solo POST /sesiones (iniciar sesión) y POST /usuarios (registrarse). Todas las demás exigen el encabezado Authorization: Bearer <token>.

- El token es un JWT firmado, emitido por el servicio de Usuarios, con las reclamaciones sub (identificador del usuario), rol, iat y exp (vigencia propuesta de 1 hora).
- El Gateway valida la firma y la vigencia y reenvía a los servicios la identidad y el rol del usuario. Cada servicio autoriza según el rol y la propiedad del registro («propias» o «su laboratorio»), como indica la columna Acceso del catálogo.
- Sin token válido la respuesta es 401; con un rol sin permiso, 403.
- Rutas internas (/internal/v1/...): solo entre servicios. El Gateway no las publica, solo se alcanzan dentro de la red de Docker Compose y exigen el encabezado X-Service-Token, cuyo valor viene de una variable de entorno y no del repositorio.
- El Gateway limita la cantidad de peticiones por cliente (429), sobre todo en el inicio de sesión.
- Las contraseñas nunca se devuelven en ninguna respuesta ni se escriben en los registros.

3. Recursos principales

Recurso	Ruta base	Qué representa	Servicio · entidad (Entrega 2)
sesiones	/sesiones	Sesión de acceso. Crear una sesión devuelve el token; no se guarda estado.	Usuarios
usuarios	/usuarios	Personas que usan el sistema.	Usuarios · Usuario, Rol
sanciones	/usuarios/{id}/sanciones	Bloqueos temporales de un usuario (subrecurso).	Usuarios · Sanción
laboratorios	/laboratorios	Espacios físicos con equipo a cargo.	Inventario · Laboratorio
categorias	/categorias	Clasificación de recursos (equipo o espacio).	Inventario · Categoría
recursos	/recursos	Equipos y espacios prestables, con su estado administrativo.	Inventario · Recurso
disponibilidad	/disponibilidad	Recursos libres en un horario. Es solo lectura y se calcula, no se guarda.	Reservas
reservas	/reservas	Solicitud de un recurso en un horario.	Reservas · Reserva
prestamos	/prestamos	Entrega y devolución reales de un recurso.	Reservas · Préstamo
incidencias	/incidencias	Daños, faltantes y retrasos.	Reservas · Incidencia
notificaciones	/notificaciones	Avisos del usuario autenticado.	Notificaciones · Notificación

Recurso	Ruta base	Qué representa	Servicio · entidad (Entrega 2)
reportes	/reportes/…	Resúmenes de uso, ocupación y retrasos. Solo lectura.	Reportes

4. Catálogo de endpoints

Se documentan 33 endpoints públicos. «Req.» indica el requisito funcional de la Entrega 2 que atiende cada uno.

Usuarios y Autenticación (6 endpoints)

#	Método	Ruta	Descripción	Acceso	Req.
E-01	POST	`/sesiones`	Iniciar sesión y obtener token	Público	RF-01
E-02	POST	`/usuarios`	Registrar un usuario	Público	RF-01
E-03	GET	`/usuarios`	Listar usuarios	Administrador	RF-02
E-04	GET	`/usuarios/{usuarioId}`	Consultar un usuario	Administrador; el propio usuario	RF-02
E-05	PATCH	`/usuarios/{usuarioId}`	Cambiar rol, estado o nombre	Administrador; el propio usuario (solo nombre)	RF-02
E-06	GET	`/usuarios/{usuarioId}/sanciones`	Consultar sanciones de un usuario	Administrador; el propio usuario	RF-07

Inventario (12 endpoints)

#	Método	Ruta	Descripción	Acceso	Req.
E-07	GET	`/laboratorios`	Listar laboratorios	Autenticado	RF-03
E-08	POST	`/laboratorios`	Crear un laboratorio	Administrador	RF-03
E-09	GET	`/laboratorios/{laboratorioId}`	Consultar un laboratorio	Autenticado	RF-03
E-10	PUT	`/laboratorios/{laboratorioId}`	Reemplazar los datos de un laboratorio	Administrador	RF-03
E-11	GET	`/categorias`	Listar categorías	Autenticado	RF-03
E-12	POST	`/categorias`	Crear una categoría	Administrador	RF-03
E-13	GET	`/recursos`	Listar y filtrar recursos	Autenticado	RF-03
E-14	POST	`/recursos`	Crear un recurso	Encargado (su laboratorio); administrador	RF-03
E-15	GET	`/recursos/{recursoId}`	Consultar un recurso	Autenticado	RF-03

#	Método	Ruta	Descripción	Acceso	Req.
E-16	PUT	`/recursos/{recursoId}`	Reemplazar los datos de un recurso	Encargado (su laboratorio); administrador	RF-03
E-17	PATCH	`/recursos/{recursoId}`	Cambiar el estado (activo, mantenimiento, baja)	Encargado (su laboratorio); administrador	RF-03
E-18	DELETE	`/recursos/{recursoId}`	Dar de baja un recurso (baja lógica)	Encargado (su laboratorio); administrador	RF-03

Reservas y Préstamos (10 endpoints)

#	Método	Ruta	Descripción	Acceso	Req.
E-19	GET	`/disponibilidad`	Consultar recursos disponibles en un horario	Autenticado	RF-04
E-20	POST	`/reservas`	Crear una reserva	Alumno, docente; encargado (mostrador)	RF-05
E-21	GET	`/reservas`	Consultar historial de reservas	Alumno, docente (propias); encargado (su laboratorio); administrador	RF-09
E-22	GET	`/reservas/{reservaId}`	Consultar una reserva	Dueño de la reserva; encargado; administrador	RF-09
E-23	PATCH	`/reservas/{reservaId}`	Modificar el horario o cancelar la reserva	Dueño de la reserva; encargado; administrador	RF-05
E-24	POST	`/prestamos`	Registrar la entrega del recurso	Encargado (su laboratorio); administrador	RF-06
E-25	GET	`/prestamos/{prestamoId}`	Consultar un préstamo	Encargado; administrador; dueño de la reserva	RF-06
E-26	PATCH	`/prestamos/{prestamoId}`	Registrar la devolución y su estado	Encargado (su laboratorio); administrador	RF-06
E-27	POST	`/incidencias`	Registrar una incidencia	Encargado (su laboratorio); administrador	RF-07
E-28	GET	`/incidencias`	Listar incidencias	Encargado (su laboratorio); administrador	RF-07

Notificaciones (2 endpoints)

#	Método	Ruta	Descripción	Acceso	Req.
E-29	GET	`/notificaciones`	Consultar las notificaciones propias	Autenticado (solo las propias)	RF-08
E-30	PATCH	`/notificaciones/{notificacionId}`	Marcar una notificación como leída	Dueño de la notificación	RF-08

Reportes (3 endpoints)

#	Método	Ruta	Descripción	Acceso	Req.
E-31	GET	`/reportes/uso`	Reporte de uso de recursos	Encargado (su laboratorio); administrador	RF-10
E-32	GET	`/reportes/ocupacion`	Reporte de ocupación por laboratorio	Encargado (su laboratorio); administrador	RF-10
E-33	GET	`/reportes/retrasos`	Reporte de retrasos e incidencias	Encargado (su laboratorio); administrador	RF-10

4.1 Rutas internas entre servicios

No se publican en el Gateway. Salen del mapa de comunicación de la Entrega 2 (llamadas ④, ⑤ y ⑥). Para consultas de lectura, los servicios también pueden usar las rutas públicas con el encabezado X-Service-Token.

#	Método	Ruta	Origen	Para qué
I-01	GET	`/internal/v1/usuarios/{usuarioId}/estado`	Reservas → Usuarios	Estado de cuenta, rol, correo y sanción vigente
I-02	POST	`/internal/v1/sanciones`	Reservas → Usuarios	Crear una sanción por retraso
I-03	PATCH	`/internal/v1/recursos/{recursoId}`	Reservas → Inventario	Poner un recurso en mantenimiento tras una devolución dañada
I-04	POST	`/internal/v1/notificaciones`	Reservas → Notificaciones	Registrar un aviso para un usuario

5. Parámetros

Parámetros de ruta (path)

Parámetro	Tipo	Descripción	Endpoints
`usuarioId`	entero ≥ 1	Identificador del usuario	E-04, E-05, E-06

Parámetro	Tipo	Descripción	Endpoints
`laboratorioId`	entero ≥ 1	Identificador del laboratorio	E-09, E-10
`recursoId`	entero ≥ 1	Identificador del recurso	E-15 a E-18
`reservaId`	entero ≥ 1	Identificador de la reserva	E-22, E-23
`prestamoId`	entero ≥ 1	Identificador del préstamo	E-25, E-26
`notificacionId`	entero ≥ 1	Identificador de la notificación	E-30

Parámetros de consulta (query)

Parámetro	Tipo	Descripción	Endpoints
`page, limit`	entero	Página (desde 1) y elementos por página (máximo 100)	Todas las listas
`inicio, fin`	fecha-hora	Horario consultado. Obligatorios; fin posterior a inicio	E-19
`desde, hasta`	fecha-hora	Filtro por periodo; en los reportes son obligatorios	E-21, E-28, E-31 a E-33
`tipo`	enumeración	equipo o espacio (recursos); dano, faltante o retraso (incidencias)	E-11, E-13, E-19, E-28
`estado`	enumeración	Estado del recurso, la reserva, la incidencia o el usuario	E-03, E-13, E-21, E-28
`laboratorioId, categoriaId`	entero	Filtra por laboratorio o categoría	E-13, E-19, E-31 a E-33
`usuarioId, recursoId`	entero	Filtra por usuario o recurso	E-21, E-28
`rol`	enumeración	alumno, docente, encargado o administrador	E-03
`q`	texto	Búsqueda por nombre	E-03, E-13
`leida`	booleano	Notificaciones leídas o no leídas	E-29

6. Contratos JSON

Cada contrato muestra la petición y la respuesta de una operación. Los ejemplos usan datos de prueba. Las propiedades de cada cuerpo están definidas en el borrador de OpenAPI.

C-01 · Iniciar sesión (E-01)

`POST /api/v1/sesiones`

Cuerpo de la petición

```
{
  "correo": "ana.lopez@ejemplo.edu.mx",
  "contrasena": "Contrasena#2026"
}
```

Respuesta · 201 Created

```
{
  "tokenAcceso": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tipo": "Bearer",
  "expiraEn": 3600,
  "usuario": { "id": 12, "nombre": "Ana López Ruiz", "rol": "alumno" }
}
```

C-02 · Registrar un usuario (E-02)

`POST /api/v1/usuarios`

Cuerpo de la petición

```
{
  "nombre": "Ana López Ruiz",
  "correo": "ana.lopez@ejemplo.edu.mx",
  "matricula": "023000123",
  "contrasena": "Contrasena#2026",
  "rol": "alumno"
}
```

Respuesta · 201 Created

```
{
  "id": 12,
  "nombre": "Ana López Ruiz",
  "correo": "ana.lopez@ejemplo.edu.mx",
  "matricula": "023000123",
  "rol": "alumno",
  "estado": "activo",
  "creadoEn": "2026-11-02T18:30:00-06:00"
}
```

C-03 · Crear un recurso (E-14)

`POST /api/v1/recursos`

Cuerpo de la petición

```
{
  "nombre": "Osciloscopio digital de 4 canales",
  "codigoInventario": "LAB-ELEC-0015",
  "categoriaId": 3,
  "laboratorioId": 2,
  "descripcion": "100 MHz"
}
```

Respuesta · 201 Created

```
{
  "id": 15,
  "nombre": "Osciloscopio digital de 4 canales",
  "codigoInventario": "LAB-ELEC-0015",
  "categoriaId": 3,
  "laboratorioId": 2,
  "descripcion": "100 MHz",
  "estado": "activo"
}
```

C-04 · Consultar disponibilidad (E-19)

`GET /api/v1/disponibilidad?inicio=2026-11-03T10:00:00-06:00&fin=2026-11-03T12:00:00-06:00&tipo=equipo&laboratorioId=2&page=1&limit=20`

Respuesta · 200 OK

```
{
  "data": [
    {
      "recursoId": 15,
      "nombre": "Osciloscopio digital de 4 canales",
      "tipo": "equipo",
      "laboratorioId": 2,
      "inicio": "2026-11-03T10:00:00-06:00",
      "fin": "2026-11-03T12:00:00-06:00"
    },
    {
      "recursoId": 18,
      "nombre": "Multímetro de banco",
      "tipo": "equipo",
      "laboratorioId": 2,
      "inicio": "2026-11-03T10:00:00-06:00",
      "fin": "2026-11-03T12:00:00-06:00"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 2 }
}
```

C-05 · Crear una reserva (E-20)

`POST /api/v1/reservas`

Cuerpo de la petición

```
{
  "recursoId": 15,
  "inicio": "2026-11-03T10:00:00-06:00",
  "fin": "2026-11-03T12:00:00-06:00"
}
```

Respuesta · 201 Created

```
{
  "id": 31,
  "usuarioId": 12,
  "recursoId": 15,
  "inicio": "2026-11-03T10:00:00-06:00",
  "fin": "2026-11-03T12:00:00-06:00",
  "estado": "confirmada",
  "creadaEn": "2026-11-02T18:35:00-06:00"
}
```

C-06 · Cancelar una reserva (E-23)

`PATCH /api/v1/reservas/31`

Cuerpo de la petición

```
{ "estado": "cancelada" }
```

Respuesta · 200 OK

```
{
  "id": 31,
  "usuarioId": 12,
  "recursoId": 15,
  "inicio": "2026-11-03T10:00:00-06:00",
  "fin": "2026-11-03T12:00:00-06:00",
  "estado": "cancelada",
  "creadaEn": "2026-11-02T18:35:00-06:00"
}
```

C-07 · Registrar la entrega (E-24)

`POST /api/v1/prestamos`

Cuerpo de la petición

```
{
  "reservaId": 31,
  "observaciones": "Se entrega con puntas y cable de alimentación"
}
```

Respuesta · 201 Created

```
{
  "id": 8,
  "reservaId": 31,
  "entregadoPor": 3,
  "fechaEntrega": "2026-11-03T10:02:00-06:00",
  "observaciones": "Se entrega con puntas y cable de alimentación"
}
```

C-08 · Registrar la devolución (dañado y con retraso) (E-26)

`PATCH /api/v1/prestamos/8`

Cuerpo de la petición

```
{
  "estadoDevolucion": "danado",
  "observaciones": "La pantalla presenta una raya"
}
```

Respuesta · 200 OK

```
{
  "id": 8,
  "reservaId": 31,
  "entregadoPor": 3,
  "fechaEntrega": "2026-11-03T10:02:00-06:00",
  "recibidoPor": 3,
  "fechaDevolucion": "2026-11-03T12:40:00-06:00",
  "estadoDevolucion": "danado",
  "observaciones": "La pantalla presenta una raya",
  "efectos": {
    "retraso": true,
    "incidenciasCreadas": [7, 8],
    "recursoEstado": "mantenimiento",
    "sancionCreada": 2
  }
}
```

C-09 · Historial de reservas con paginación (E-21)

`GET /api/v1/reservas?usuarioId=12&estado=finalizada&page=1&limit=2`

Respuesta · 200 OK


```
{
  "data": [
    {
      "id": 30,
      "usuarioId": 12,
      "recursoId": 18,
      "inicio": "2026-10-27T09:00:00-06:00",
      "fin": "2026-10-27T11:00:00-06:00",
      "estado": "finalizada",
      "creadaEn": "2026-10-26T17:00:00-06:00"
    },
    {
      "id": 22,
      "usuarioId": 12,
      "recursoId": 15,
      "inicio": "2026-10-20T10:00:00-06:00",
      "fin": "2026-10-20T12:00:00-06:00",
      "estado": "finalizada",
      "creadaEn": "2026-10-19T16:20:00-06:00"
    }
  ],
  "meta": { "page": 1, "limit": 2, "total": 5 }
}
```


C-10 · Reporte de uso de recursos (E-31)

`GET /api/v1/reportes/uso?desde=2026-10-01T00:00:00-06:00&hasta=2026-10-31T23:59:59-06:00&laboratorioId=2`

Respuesta · 200 OK


```
{
  "desde": "2026-10-01T00:00:00-06:00",
  "hasta": "2026-10-31T23:59:59-06:00",
  "recursos": [
    {
      "recursoId": 15,
      "nombre": "Osciloscopio digital de 4 canales",
      "totalPrestamos": 9,
      "horasUso": 21.5
    },
    {
      "recursoId": 18,
      "nombre": "Multímetro de banco",
      "totalPrestamos": 6,
      "horasUso": 12
    }
  ]
}
```

7. Códigos HTTP

Código	Nombre	Cuándo se usa en PrestaLab	Ejemplo
200	OK	La consulta o la modificación se realizó y la respuesta trae el recurso.	GET /recursos; PATCH /reservas/{id}
201	Created	Se creó un recurso. La respuesta trae el recurso y el encabezado Location.	POST /reservas; POST /usuarios
204	No Content	La operación se realizó y no hay cuerpo que devolver.	DELETE /recursos/{id}
400	Bad Request	La petición está mal formada: JSON roto, campo obligatorio ausente, tipo incorrecto o parámetro fuera de rango.	POST /reservas sin el campo fin
401	Unauthorized	Falta el token, expiró o no es válido; o las credenciales del inicio de sesión son incorrectas.	Cualquier ruta protegida sin token
403	Forbidden	El usuario está autenticado, pero su rol o su propiedad sobre el registro no permiten la operación.	PATCH /usuarios/{id} hecho por un alumno
404	Not Found	El recurso indicado en la ruta no existe.	GET /recursos/9999
409	Conflict	La operación choca con el estado actual: reserva traslapada, correo o código de inventario repetidos, estado incompatible.	POST /reservas en un horario ocupado
422	Unprocessable Content	La petición está bien formada, pero viola una regla: límite de reservas, usuario sancionado, recurso no reservable, fuera de plazo.	POST /reservas de un usuario sancionado
429	Too Many Requests	El Gateway limitó la cantidad de peticiones de un mismo cliente.	POST /sesiones repetido muchas veces
500	Internal Server Error	Falla inesperada del servicio. El mensaje no revela detalles internos.	Cualquier ruta
503	Service Unavailable	Un servicio del que depende la operación no respondió.	POST /reservas si Inventario no responde

Regla para distinguir errores de cliente: 400 es una petición mal formada; 409 es un choque con el estado actual de los datos; 422 es una petición correcta que viola una regla de negocio.

8. Manejo de errores

8.1 Formato estándar

Toda respuesta de error (4xx y 5xx) usa el mismo cuerpo, inspirado en los detalles de problema de HTTP (RFC 9457). Cada error lleva un código de aplicación estable, que el consumidor puede usar sin depender del texto del mensaje.

```
{
  "error": {
    "codigo": "RESERVA_TRASLAPADA",
    "mensaje": "El recurso ya está reservado en ese horario.",
    "detalles": [
      {
        "campo": "inicio",
        "problema": "Se traslapa con la reserva 31"
      }
    ],
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

Campo	Obligatorio	Descripción
codigo	Sí	Código de aplicación en mayúsculas; no cambia entre versiones.
mensaje	Sí	Texto legible para la persona. Nunca incluye trazas, nombres de tablas ni datos internos.
detalles	No	Lista de { campo, problema } para errores de validación o conflicto.
ruta	Sí	Ruta solicitada.
fecha	Sí	Momento del error (ISO 8601).
idPeticion	Sí	Identificador asignado por el Gateway, para rastrear la petición en los registros.

8.2 Códigos de error de aplicación

Código	HTTP	Cuándo ocurre
VALIDACION_FALLIDA	400	Un campo falta o tiene un formato inválido. Se detalla en «detalles».
JSON_INVALIDO	400	El cuerpo de la petición no es JSON válido.
TOKEN_INVALIDO	401	Token ausente, vencido o alterado.
CREDENCIALES_INVALIDAS	401	Correo o contraseña incorrectos (sin indicar cuál).
PERMISO_INSUFICIENTE	403	El rol no permite la operación.
NO_ENCONTRADO	404	El recurso de la ruta no existe.

Código	HTTP	Cuándo ocurre
CORREO_REPETIDO	409	Ya existe un usuario con ese correo.
CODIGO_INVENTARIO_REPETIDO	409	Ya existe un recurso con ese código de inventario.
RESERVA_TRASLAPADA	409	El recurso ya está reservado en ese horario.
ESTADO_INCOMPATIBLE	409	El estado actual no permite la operación (por ejemplo, entregar una reserva cancelada).
USUARIO_SANCIONADO	422	El usuario tiene una sanción vigente.
LIMITE_RESERVAS_EXCEDIDO	422	El usuario alcanzó el límite de reservas vigentes de su rol.
RECURSO_NO_RESERVABLE	422	El recurso no está en estado activo.
FUERA_DE_PLAZO	422	Ya no se puede cancelar o modificar la reserva.
DEMASIADAS_PETICIONES	429	Se superó el límite de peticiones del Gateway.
ERROR_INTERNO	500	Falla inesperada.
SERVICIO_NO_DISPONIBLE	503	Un servicio del que depende la operación no respondió.

8.3 Ejemplos de errores

X-01 · 400 · Campo obligatorio ausente

`POST /api/v1/reservas`

```
{
  "error": {
    "codigo": "VALIDACION_FALLIDA",
    "mensaje": "Los datos enviados no son válidos.",
    "detalles": [{ "campo": "fin", "problema": "Es obligatorio" }],
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

X-02 · 401 · Token vencido

`GET /api/v1/reservas`

```
{
  "error": {
    "codigo": "TOKEN_INVALIDO",
    "mensaje": "El token no es válido o expiró. Inicie sesión de nuevo.",
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

X-03 · 403 · Alumno intenta cambiar un rol

`PATCH /api/v1/usuarios/5`


```
{
  "error": {
    "codigo": "PERMISO_INSUFICIENTE",
    "mensaje": "Su rol no permite realizar esta operación.",
    "ruta": "/api/v1/usuarios/5",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```


X-04 · 404 · Recurso inexistente

`GET /api/v1/recursos/9999`


```
{
  "error": {
    "codigo": "NO_ENCONTRADO",
    "mensaje": "No existe un recurso con el identificador 9999.",
    "ruta": "/api/v1/recursos/9999",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```


X-05 · 409 · Horario ya reservado

`POST /api/v1/reservas`


```
{
  "error": {
    "codigo": "RESERVA_TRASLAPADA",
    "mensaje": "El recurso ya está reservado en ese horario.",
    "detalles": [
      {
        "campo": "inicio",
        "problema": "Se traslapa con la reserva 31"
      }
    ],
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

X-06 · 409 · Correo ya registrado

`POST /api/v1/usuarios`

```
{
  "error": {
    "codigo": "CORREO_REPETIDO",
    "mensaje": "Ya existe un usuario con ese correo.",
    "detalles": [{ "campo": "correo", "problema": "Ya está registrado" }],
    "ruta": "/api/v1/usuarios",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

X-07 · 422 · Límite de reservas alcanzado

`POST /api/v1/reservas`

```
{
  "error": {
    "codigo": "LIMITE_RESERVAS_EXCEDIDO",
    "mensaje": "Alcanzó el límite de reservas vigentes de su rol (2).",
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

X-08 · 503 · Inventario no responde

`POST /api/v1/reservas`

```
{
  "error": {
    "codigo": "SERVICIO_NO_DISPONIBLE",
    "mensaje": "No fue posible verificar el recurso. Intente de nuevo en unos minutos.",
    "ruta": "/api/v1/reservas",
    "fecha": "2026-11-02T18:40:12-06:00",
    "idPeticion": "req-7f3a91c2"
  }
}
```

9. Diagramas de flujo de operaciones completas

Los dos diagramas muestran, paso a paso, qué decide el sistema y con qué código HTTP responde en cada rama. Sirven también como guía para implementar las reglas en las semanas 5 y 6.

9.1 Crear una reserva (E-20)

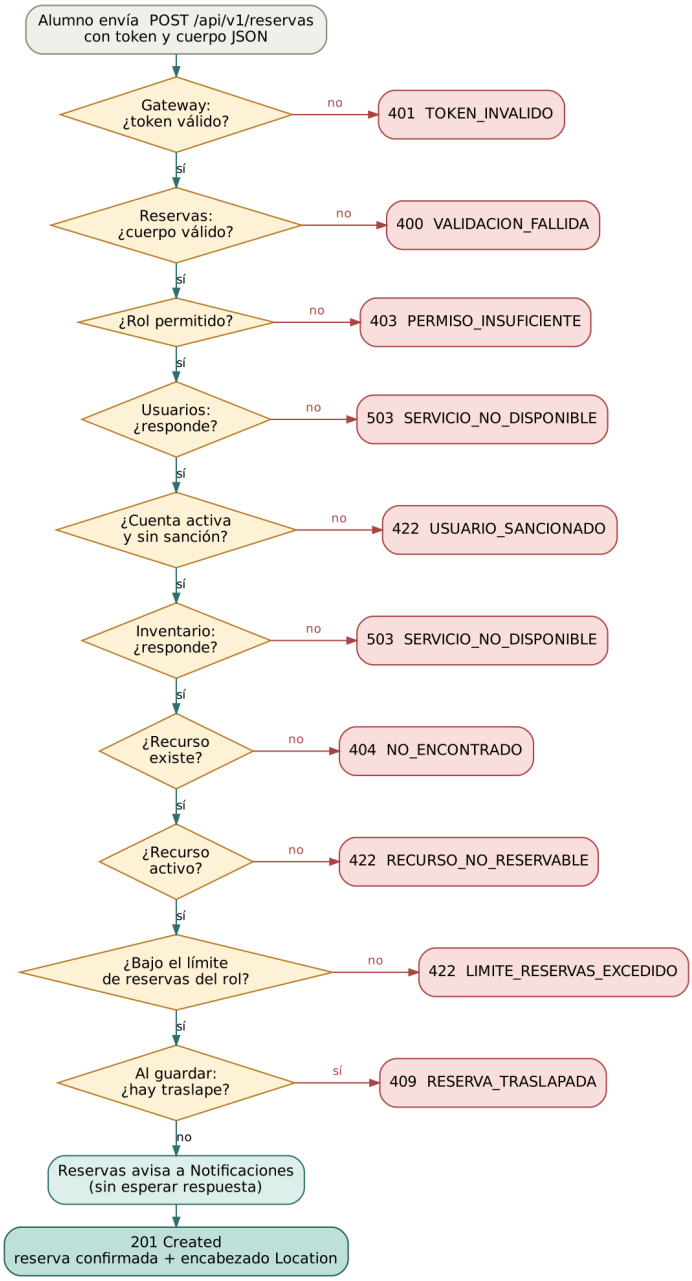


Figura 1. Flujo de POST /api/v1/reservas.

9.2 Registrar la devolución de un préstamo (E-26)

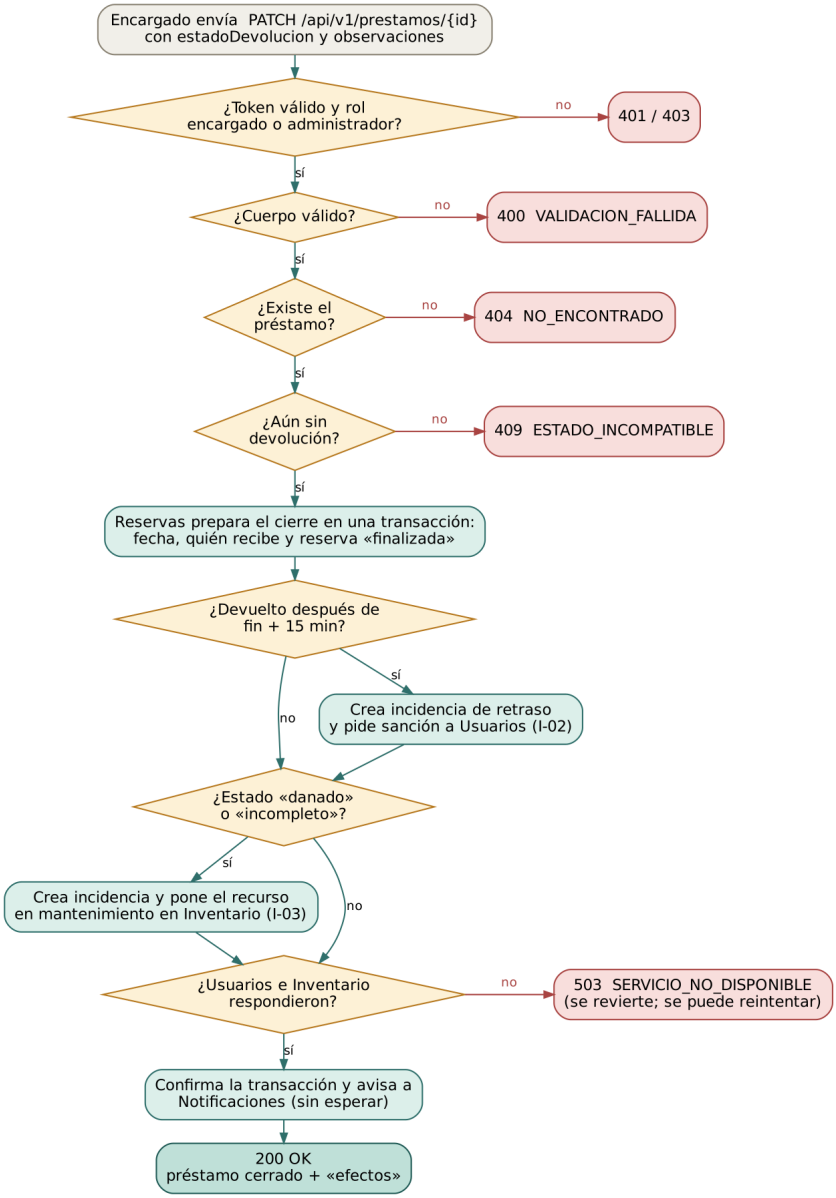


Figura 2. Flujo de PATCH /api/v1/prestamos/{prestamoId}.

10. Borrador OpenAPI

Se entrega el archivo openapi.yaml (OpenAPI 3.0.3) con la misma información de este documento: 33 operaciones en 20 rutas, 38 esquemas de datos, parámetros reutilizables, el esquema de seguridad Bearer JWT, las respuestas de error estándar y los ejemplos de los contratos de la sección 6. El archivo pasó la validación de especificación OpenAPI.

- Para verlo, se puede pegar en el editor de Swagger (editor.swagger.io) o importar en Postman.
- Las rutas internas no se incluyen, porque no son parte del contrato público.

- En la Entrega 7 se publicará con Swagger UI y se mantendrá sincronizado con el código.

11. Verificación de mínimos técnicos y criterios

Elemento	Mínimo pedido	Entregado	Dónde
Recursos	5	12	Sección 3
Endpoints documentados	20	33 (más 4 internos)	Sección 4
Ejemplos JSON de request	5	7	Sección 6
Ejemplos JSON de response	5	10	Sección 6
Códigos HTTP explicados	8	12	Sección 7
Casos de error	5	8	Sección 8.3

Trabajo requerido	Dónde se cumple
Identificar los recursos principales de la API	Sección 3
Diseñar rutas REST claras y consistentes	Secciones 1 y 4
Asignar métodos HTTP correctos	Secciones 1 y 4
Definir path parameters y query parameters	Sección 5
Definir request body y response body	Sección 6
Definir códigos HTTP de éxito y error	Sección 7
Definir un formato estándar de errores	Sección 8
Definir la autenticación prevista	Sección 2
Definir el versionado inicial de la API	Sección 1

Entregable obligatorio	Dónde se cumple
Catálogo de endpoints	Sección 4
Contratos JSON	Sección 6
Tabla de códigos HTTP	Sección 7
Ejemplos de errores	Sección 8.3
Diagrama de flujo de al menos una operación completa	Sección 9 (dos operaciones)

Entregable obligatorio	Dónde se cumple
Primer borrador OpenAPI/Swagger	Archivo openapi.yaml (sección 10)

Criterio de aceptación	Cómo se cumple
Los endpoints representan recursos y no acciones arbitrarias	Rutas con sustantivos; las acciones son recursos (sesiones, préstamos) o cambios de estado (PATCH).
GET, POST, PUT/PATCH y DELETE se usan de forma coherente	Se usan los cinco: 17 GET, 8 POST, 2 PUT, 5 PATCH y 1 DELETE, con la semántica de la sección 1.
Los contratos permiten saber qué enviar y recibir	Sección 6 y archivo openapi.yaml.
La API contempla errores, no solo casos exitosos	Cada endpoint lista sus errores; secciones 7 y 8.

12. Fuentes y referencias

- Proyecto_SOA.pdf: lineamientos del Proyecto Integrador, Entrega 3.
- Entrega 2 del equipo: requisitos, roles, servicios y mapa de comunicación.
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (tesis doctoral). University of California, Irvine.
- Fielding, R., Nottingham, M. y Reschke, J. (2022). HTTP Semantics. RFC 9110. https://www.rfc-editor.org/rfc/rfc9110
- Jones, M., Bradley, J. y Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519
- Nottingham, M. y Wilde, E. (2023). Problem Details for HTTP APIs. RFC 9457. https://www.rfc-editor.org/rfc/rfc9457
- OpenAPI Initiative. OpenAPI Specification v3.0.3. https://spec.openapis.org/oas/v3.0.3

Anexo A. Bitácora de cambios

Semana	Cambio realizado	Motivo	Responsable(s)	Evidencia
1	Versión inicial de la propuesta	Primera entrega	Equipo	Documento de la Entrega 1
2	Estado administrativo separado de la disponibilidad	Evitar responsabilidades duplicadas entre Inventario y Reservas	Equipo	Documento de la Entrega 2
2	10 RF, 9 RNF, matriz de permisos, servicios y diagrama	Contenido de la Entrega 2	Equipo	Documento de la Entrega 2
3	33 endpoints REST y 4 rutas internas	Contenido de la Entrega 3	Equipo	Secciones 3 y 4
3	Formato único de errores y regla 400 / 409 / 422	Respuestas de error consistentes	Equipo	Secciones 7 y 8
3	Disponibilidad como recurso derivado (/disponibilidad)	Separar la consulta de horarios del catálogo	Equipo	Sección 3
3	Borrador OpenAPI 3.0.3 validado	Contrato legible por herramientas	Equipo	Archivo openapi.yaml